



# Security Assessment



# Aave Stable Vaults

May 2026

Prepared for Aave Labs

## Table of Contents

|                                                                                                                                |           |
|--------------------------------------------------------------------------------------------------------------------------------|-----------|
| <b>Project Summary.....</b>                                                                                                    | <b>3</b>  |
| Project Scope.....                                                                                                             | 3         |
| Project Overview.....                                                                                                          | 3         |
| Protocol Overview.....                                                                                                         | 4         |
| <b>Assessment Methodology.....</b>                                                                                             | <b>6</b>  |
| <b>Threat and Security Overview.....</b>                                                                                       | <b>7</b>  |
| Findings Summary.....                                                                                                          | 10        |
| Severity Matrix.....                                                                                                           | 10        |
| <b>Detailed Findings.....</b>                                                                                                  | <b>11</b> |
| Low Severity Issues.....                                                                                                       | 12        |
| L-01: Users can steal 1 wei from the vault by transferring shares.....                                                         | 12        |
| L-02: Override mode grants the rebalancer unrestricted access to the SlippageCoverageVault risking all funds in the vault..... | 14        |
| Informational Severity Issues.....                                                                                             | 17        |
| I-01: Admin can extract yield from users by transferring them between vaults.....                                              | 17        |
| I-02: Misleading ExceedsWindowCap error when window is unconfigured.....                                                       | 19        |
| I-03: SlippageToleranceTooHigh check is bypassable and does not bound actual slippage.....                                     | 20        |
| <b>Disclaimer.....</b>                                                                                                         | <b>21</b> |
| <b>About Certora.....</b>                                                                                                      | <b>21</b> |

# Project Summary

## Project Scope

| Project Name       | Repository Link                                                                         | Commit Hash                                                                  | Platform |
|--------------------|-----------------------------------------------------------------------------------------|------------------------------------------------------------------------------|----------|
| Aave Stable Vaults | <a href="https://github.com/aave/stable-vault">https://github.com/aave/stable-vault</a> | <a href="#">1c179e9</a><br><a href="#">3afa6b4</a><br><a href="#">PR#354</a> | Solidity |

## Project Overview

This document describes the security review of **AAVE Stable Vault**. The work was undertaken from **May 18th, 2026** to **May 22th, 2026**.

The following contract list is included in our scope:

```
stable-vault/src/bridging/ccip/CcipAdapter.sol
stable-vault/src/bridging/BaseBridgeAdapter.sol
stable-vault/src/bridging/adi/AdiAdapter.sol
stable-vault/src/core/accounting/AccountingChainGateway.sol
stable-vault/src/core/accounting/StableVault.sol
stable-vault/src/core/accounting/FundsHandler.sol
stable-vault/src/core/earning/EarningChainGateway.sol
stable-vault/src/core/ious/IouToken.sol
stable-vault/src/core/ious/IouTokenManager.sol
stable-vault/src/core/Allocator.sol
stable-vault/src/core/BaseChainGateway.sol
stable-vault/src/core/LocalBalanceAggregator.sol
stable-vault/src/libraries/AssetLib.sol
stable-vault/src/libraries/MathLib.sol
stable-vault/src/libraries/RateLimitBucketLib.sol
stable-vault/src/misc/Multicall.sol
```

```
stable-vault/src/misc/RescuableNative.sol
stable-vault/src/misc/RescuableToken.sol
stable-vault/src/misc/TransferHelperClient.sol
stable-vault/src/periphery/AssetRegistry.sol
stable-vault/src/periphery/Swapper.sol
stable-vault/src/periphery/SlippageCoverageVault.sol
stable-vault/src/periphery/PolicyRegistry.sol
stable-vault/src/periphery/TransferHelper.sol
stable-vault/src/periphery/WithdrawalPolicy.sol
stable-vault/src/periphery/OwnedMulticall.sol
stable-vault/src/periphery/EarningChainStateSchemaV1.sol
stable-vault/src/periphery/EarningChainStateProvider.sol
stable-vault/src/policies/DepositPolicy.sol
stable-vault/src/policies/FundsBridgingPolicy.sol
stable-vault/src/policies/WithdrawalExecutionPolicy.sol
stable-vault/src/policies/base/GlobalRateLimitedPolicy.sol
stable-vault/src/types/Constants.sol
stable-vault/src/types/Errors.sol
stable-vault/src/oracles/balance/ChainBalanceOracle.sol
stable-vault/src/oracles/balance/ChainlinkChainBalanceOracleAdapter.sol
stable-vault/src/oracles/balance/ChainlinkL2ChainBalanceOracleAdapter.sol
stable-vault/src/oracles/common/L2ChainlinkOracleAdapter.sol
stable-vault/src/oracles/price/ChainlinkL2PriceOracleAdapter.sol
stable-vault/src/oracles/price/ChainlinkPriceOracleAdapter.sol
stable-vault/src/oracles/price/PriceOracle.sol
```

The team performed a manual audit of all the files in scope. During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

## Protocol Overview

Stable Vaults is a semi-fixed rate vault protocol developed by Aave Labs. It allows users to deposit assets into "SubVaults" that offer a specific per-second interest rate, enabling predictable yield generation. Deposited funds are allocated to yield-generating strategies (e.g. Aave). The system tracks each user's original principal independently from accrued interest. In

the event the vault cannot meet its total interest obligations, users are prioritized to withdraw at least their original deposit amount.

The system allows users to deposit whitelisted USD denominated stable coins on the accounting (canonical) chain. The depositor is minted vault shares, which are not transferable; funds are automatically deployed to the default strategy, when set.

The withdrawal process has two steps:

1. Vault share holders first request withdrawal of principal and interest – they are only allowed to withdraw interest if protocol is solvent. Withdrawal requests burn vault shares and mint IOU tokens. This stops interest accrual and locks the USD value of the withdrawal.
2. IOU token holders can then redeem it for whitelisted USD denominated stable coins, minus a withdrawal fee. This can be done on the canonical chain; or IOUs can be bridged to any of the earning chains and redeemed there. There is a default withdrawal fee charged from all withdrawals – this fee can be overridden if the user provides a signed message from the whitelisted signer. The withdrawal attempts to use idle funds not deployed to any strategy; if insufficient, it attempts to withdraw from the default strategy; if insufficient, from other strategies.

The system considers one unit of whitelisted stable coins to be worth 1 USD, and does not track deposit per asset back to users.

Deposits accrue a fixed-rate yield, set by governance for each user.

The funds deposited by users are automatically deployed to the default strategy. Governance whitelisted addresses can manage the funds, swapping the tokens held; or rebalancing the strategies to which these tokens are deployed.

# Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

# Threat and Security Overview

## Scope and Trust Assumptions

The review focused on logic errors and vulnerabilities in the smart contract code, including accounting-chain and earning-chain interactions, vault share and IOU accounting consistency, interest accrual correctness, defensive rounding, ERC-4626 strategy integration, withdrawal solvency, and asset trust/distrust handling. The review was conducted through manual analysis, invariants, and hypotheses.

The following are treated as trust assumptions, that must hold for the protocol to operate safely:

- Blockchain liveness and integrity: blocks are produced continuously, valid transactions are accepted, invalid ones rejected, fees remain reasonable.
- Bridge and cross-chain messaging integrity: the underlying messaging layers deliver messages reliably and in order, and do not drop, forge, or maliciously reorder cross-chain calls. The audit verified the protocol's integration with these layers only, not their internal security.
- Oracle correctness: the price feeds used for peg and depeg detection are timely, accurate, and manipulation-resistant. The audit verified integration only, not feed quality.
- Strategy integrity: external yield strategies (e.g., Aave) conform to the ERC-4626 standard, remain solvent, and are not malicious or arbitrarily upgradeable. The audit verified the vault's interaction with strategies, not the strategies themselves.
- Admin and governance integrity: multisig keys are secure, the execution time-lock is honored, and governance proposals are not malicious.
- Manager honesty within bounds: the semi-trusted manager performs rebalancing, rate-setting, surplus claims, and cross-chain fund movement without abusing its privileges. It is assumed unable to claim principal or cause principal loss, but capable of affecting yield and operational outcomes.
- Signer integrity: the whitelisted fee-discount signer's key is secure and is used only to sign legitimate withdrawal-fee overrides.
- Off-chain monitoring and response: monitoring systems detect depegs, anomalous balances, and malicious strategies, and operators act promptly, since deposits and withdrawals are permissionless and can deplete value before manual intervention.

## System Invariants

The system is designed to preserve the following invariants under all conditions:

- Users can always redeem IOUs for their Original Deposit.
- IOUs are minted for interest only up to trusted surplus above guaranteed obligations.
- Admin may claim surplus only, defined as total assets minus (total principal deposits + total interest accrued).
- TVL cannot be artificially inflated.

## Roles

The following roles carry distinct trust levels that must hold for the protocol to operate safely:

- Admin: fully trusted entity operating through a multisig account with an execution time-lock on each operation.
- Manager: semi-trusted entity responsible for rebalancing, setting interest rates, claiming surplus, and moving funds across chains, but unable to claim principal or cause principal loss.

## Security Considerations and Threat Analysis

**Deposit Integrity.** A deposit must never degrade system solvency or shift costs onto existing users. Threats include:

- Deposits that harm the system or other depositors under any scenario.
- Deposit of depegged assets in the absence of a price check, rendering the system insolvent.
- Deployment slippage to the default strategy borne by the vault rather than the depositor.

**Denial of Service.** Critical actions, especially withdrawals, must remain executable at all times. Threats include:

- Blocking, stalling, or forcing retries of withdrawals or other critical flows.
- Nonce or replay-based DoS of discounted-fee withdrawals (e.g., front-running to consume a single-use signed message).

**Access Control.** Only authorized roles may perform privileged actions. Threats include:

- Bypassing access restrictions on protected functions.
- Spoofing or replaying signed messages and permissions.

**Cross-Chain Integrity.** Cross-chain accounting and messaging must be tamper-resistant and reliable. Threats include:

- Manipulating total TVL via cross-chain snapshots.
- Spoofing an earning chain's reported TVL or maliciously minting IOUs through cross-chain components.
- Cross-chain transfers or messages that can be blocked, stuck, or maliciously retried.

**TVL and Interest Manipulation.** TVL and accrued interest must reflect the true economic state. Threats include:

- Artificial inflation or deflation of TVL, impacting principal and interest claims.
- Manipulating the interest-rate mechanism so the effective rate diverges from the intended rate.

**User-to-User Harm.** No user action should harm other users or elevate systemic risk. Threats include:

- Actions that damage other users' claims or increase the risk of TVL loss.

**Asset and Strategy Risk Management.** The protocol must be able to retire dangerous components. Threats include:

- Inability to remove a heavily depegging asset or a malicious strategy to limit risk and losses.
- Limited admin-controlled mechanisms to minimize exposure when an asset or strategy becomes dangerous.

## Findings Summary

The table below summarizes the findings of the review, including type and severity details.

| Severity      | Discovered | Confirmed | Fixed    |
|---------------|------------|-----------|----------|
| Critical      | –          | –         | –        |
| High          | –          | –         | –        |
| Medium        | –          | –         | –        |
| Low           | 2          | 2         | 1        |
| Informational | 3          | 3         | 1        |
| <b>Total</b>  | <b>5</b>   | <b>5</b>  | <b>2</b> |

## Severity Matrix

|                   |        |        |        |          |
|-------------------|--------|--------|--------|----------|
| <b>Impact</b>     | High   | Medium | High   | Critical |
|                   | Medium | Low    | Medium | High     |
|                   | Low    | Low    | Low    | Medium   |
|                   |        | Low    | Medium | High     |
| <b>Likelihood</b> |        |        |        |          |

# Detailed Findings

| ID                   | Title                                                                                                               | Severity      | Status       |
|----------------------|---------------------------------------------------------------------------------------------------------------------|---------------|--------------|
| <a href="#">L-01</a> | Users can steal 1 wei from the vault by transferring shares                                                         | Low           | Fixed        |
| <a href="#">L-02</a> | Override mode grants the rebalancer unrestricted access to the SlippageCoverageVault risking all funds in the vault | Low           | Acknowledged |
| <a href="#">I-01</a> | Admin can extract yield from users by transferring them between vaults                                              | Informational | Acknowledged |
| <a href="#">I-02</a> | Misleading ExceedsWindowCap error when window is unconfigured                                                       | Informational | Fixed        |
| <a href="#">I-03</a> | SlippageToleranceTooHigh check is bypassable and does not bound actual slippage                                     | Informational | Acknowledged |

## Low Severity Issues

### L-01: Users can steal 1 wei from the vault by transferring shares

|                           |               |                    |
|---------------------------|---------------|--------------------|
| Severity: Low             | Impact: Low   | Likelihood: Medium |
| Files:<br>StableVault.sol | Status: Fixed |                    |

#### Description:

When a user transfers funds via `StableVault::transfer()`, they pass an `amountRay`, which is used to calculate `guaranteedAmountRay` and `fromUserShares`.

- `guaranteedAmountRay`: as long as the user is not transferring their full balance and still has guaranteed amount left, `guaranteedAmountRay = amountRay`
- `fromUserShares`: the equivalent amount of shares of the transferred amount, rounded **up**

If the receiving user is on the same subvault as the sender, the amount of shares received equals the amount of shares sent.

That enables an attack where an user can steal from the vault, as shown in the example below:

- Assume a conversion rate of 2
- Alice deposits 6 wei, and gets
  - `positions[user].shares = 3`
  - `positions[user].originalDepositRay = 6`
- Alice transfers 3 wei to Bob, where
  - `guaranteedAmountRay = 3`

- fromUserShares = 2
- This results in:
  - Alice new balances:
    - shares = 1
    - originalDepositRay = 3
  - Bob new balances:
    - shares = 2
    - originalDepositRay = 3

Upon withdrawal, shares are valued according to the vault's conversion rate, as long as the resulting value is greater than or equal to the user's originalDepositRay.

On the scenario above:

- Bob will be able to withdraw 4 wei, which is 2 shares \* conversion rate of 2
- Alice would be able to withdraw only 2 wei, which is 1 share \* conversion rate of 2. However, that would be increased to 3 as that is her original deposit ray

Together, Alice and Bob can withdraw 7 wei, after a deposit of 6 wei, stealing 1 wei from the vault.

### Recommendations:

Alternative solutions:

- Decrease the sender's guaranteed amount by the amount actually sent  
`shares.rayMulUp(fromConversionRate)`
- Recalculate the shares received by the receiver address from the shares, rounding down

### Fix Review:

Fixed in [PR#341](#) and [PR#351](#).

L-02: Override mode grants the rebalancer unrestricted access to the SlippageCoverageVault risking all funds in the vault

| Severity: Low                                                                                            | Impact: High         | Likelihood: Low |
|----------------------------------------------------------------------------------------------------------|----------------------|-----------------|
| Files:<br>src/periphery/SlippageCoverageVault.sol<br>src/periphery/Swapper.sol<br>src/core/Allocator.sol | Status: Acknowledged |                 |

### Description:

The rate limits on slippage consumption are a great defense if the rebalancer is compromised. In an emergency, with `overrideMode` on, the rebalancer is granted the ability to use max slippage and no rate limits on any token in the vault.

The situation may require it, but it would be better if this ability is never granted to the rebalancer. Since the rebalancer is an off-chain service there is a chance it could get compromised (leaked keys or similar) and the attacker is waiting for the right moment to inflict maximum damage. The critical swap that allows any kind of slippage should only be executable by the admin/team.

There are 3 concerns:

**First concern** is if depeg happens and `overrideMode` is triggered.

`slippageCoverageVault.sweep()` has a 7 days delay so there is no way to isolate the tokens and leave only the required tokens in the SlippageCoverageVault.

If SlippageCoverageVault has:

- 1M USDC
- 1M USDT
- 1M GHO

And if 1M USD is needed to cover the slippage, `overrideMode` is turned on and 3M is put at risk instead of 1M.

If `SlippageCoverageVault` is not expected to hold millions, much less for day to day operations: USDC 20k, USDT 20k, GHO 20k and if emergency happens admin transfers 1M USDC to cover the big swap then all assets in the `SlippageCoverageVault` are at risk but the potential damage is much more limited/controlled.

**Second concern** is leaving the system in its current state and if the rebalancer is compromised or is not fully trusted, when `overrideMode` is turned on, nothing is stopping the rebalancer from swapping a healthy asset instead of the problematic one.

If Stablecoin X depegs, admin turns `overrideMode` on and expects the rebalancer to do Stablecoin X → USDC swap. Instead rebalancer swaps 1M USDT for 100k USDC. 900k USDC slippage is covered by the slippage vault. 900k USDT is sent to a malicious address. The system is still left holding the depegging Stablecoin X.

If the rebalancer is allowed to use max slippage only for **assetIn** being **Stablecoin X** this concern would be addressed.

**Third concern.** At the moment the rebalancer is trusted that it will not intentionally swap Stablecoin X for less than it can.

Assume there is 100M of Stablecoin X in the Allocator. It depegs heavily and the admin sends 20M USDC to slippage coverage vault. The admin turns `overrideMode` on and even if rebalancer can use max slippage only when **assetIn** is Stablecoin X (the recommendation from concern 2 from above) it can still swap 20M Stablecoin X for 1 wei USDC and use the whole 20M USDC in the slippage coverage vault to cover the slippage.

The admin expected to hold 100M USDC after the swap and 0 Stablecoin X but got 20M USDC and 80M Stablecoin X instead.

### Recommendations:

`Allocator._swap()` could be made to accept an optional signature that is forwarded to the Swapper and SlippageCoverageVault.

The signature commits to the exact swap parameters (swap data, max slippage, max amount pullable from coverage) and authorizes that one swap to bypass rate limits. When the signature is empty, normal rate limits are applied.

The idea is that the rebalancer is never freed from rate limits and slippage %, and despite the emergency is never given full power to drain the slippage vault.

Another approach is to allow `overrideMode` only for certain assetIn being swapped but the third concern stays in place if swap data is not enforced.

### Customer Response:

We are aware of this risk, which is inherent to the current system design. The `SlippageCoverageVault` will not hold unreasonably large balances, and all rebalancer actions will be monitored so that the rebalancer can be revoked if it is found to be acting maliciously. We accept this as a residual risk.

## Informational Severity Issues

I-01: Admin can extract yield from users by transferring them between vaults

### Files:

StableVault.sol

### Description:

Whenever a user deposits into the stable vault, they get assigned to the vault with the default rate.

If the admin wants to grant them a different rate, they can call `StableVault::setUserRate()` passing an array of users and the corresponding desired rate, which will transfer the user from their current sub vault to the sub vault with the desired rate.

Given each sub vault accrues value in a different way, shares are scoped individually for them, such that transfer the user from one sub vault to another implies converting their shares.

The share conversion is done by rounding down, as it should be, to protect the system from getting unbacked debt. However, one side effect of that is that the admin can abuse this to cause losses to users, by sequentially transferring a user between sub vaults, causing them to lose shares with each conversion.

There are two additional considerations, though:

- Shares are denominated in RAY, which massively reduces the potential harm that can be done with this exploit, since it would require  $1e27$  cycles to cause 1 full share of damage (worth \$1 around deployment and increasing at most 20% per year for the vault on the contract's maximum rate)

- On the other hand, the `StableVault::setUserRate()` function accepts an array that does not enforce user uniqueness, which makes it possible to set the same user from vault A to vault B and back several times with a low gas cost, since the next interactions will write to warm storage locations

After a user has been moved from vault A to vault B, moving them back to vault A in the same transaction would only perform 5 additional storage writes, all of them warm. Each warm write costs 2900 gas, such that each additional loop would cost 14,500 gas (plus the cheaper computations).

This allows an admin to fit over 1,000 cycles in a single transaction, which would usually cause roughly 1,000 wei of losses to the user (in RAY denomination).

While this does not look significant and does not grant a fix recommendation, we decided to include the analysis in the report for awareness and completeness that this exploit vector exists, but has no practical impact as of the intended use of the system.

### **Recommendations:**

No changes recommended.

### **Customer Response:**

Acknowledged; the finding recommends no code change, and none is required. `setUserRate()` is a restricted, access-controlled action performed only by trusted manager roles (Primer Assumption #1), whose activity is monitored off-chain and who can be revoked without timelock if found to be acting maliciously (Primer Assumption #17).

The grieving described is economically negligible: shares are denominated in RAY, so approximately  $1e27$  conversion cycles would be required to cost a user a single full share (worth roughly \$1), while a transaction fitting around 1,000 cycles inflicts only about 1,000 wei of RAY-denominated loss. We therefore accept this as an informational finding with no practical impact under the intended use of the system.

## I-02: Misleading ExceedsWindowCap error when window is unconfigured

### Files:

src/periphery/SlippageCoverageVault.sol

### Description:

`_consumeWindow` reverts with `ExceedsWindowCap` for two distinct failure modes:

```
require(window.cap > 0 && window.windowSeconds > 0, ExceedsWindowCap());  
...  
require(newConsumed <= uint256(window.cap), ExceedsWindowCap());
```

The first check fires when the window has never been configured for the asset (`cap == 0` or `windowSeconds == 0`), not because a cap was exceeded. While the natspec on `ExceedsWindowCap` documents the dual usage ("...or the window is unconfigured"), conflating the two conditions degrades observability.

### Recommendations:

Introduce a dedicated error and split the require so each failure mode has a unique selector.

### Fix Review:

Fixed in [PR#326](#).

## I-03: SlippageToleranceTooHigh check is bypassable and does not bound actual slippage

### Files:

src/periphery/Swapper.sol:88

### Description:

The check `require(slippageToleranceBps <= coverageSlippageMaxBps, SlippageToleranceTooHigh())` validates the rebalancer's stated tolerance against the vault's configured cap, not the actual slippage that occurred.

The actual slippage in bps is  $(\text{expectedAmountOut} - \text{amountOut}) * \text{MAX\_BPS} / \text{expectedAmountOut}$ , which is never directly compared against `coverageSlippageMaxBps`.

The check provides no reliable enforcement of `coverageSlippageMaxBps` as a protocol-level bound. The actual protection against excessive coverage drainage comes solely from the window and per-tx caps enforced inside `SlippageCoverageVault.pullCoverage`.

### Recommendations:

Consider checking the slippage based on the following formula:  $(\text{expectedAmountOut} - \text{amountOut}) * \text{MAX\_BPS} / \text{expectedAmountOut}$ .

### Customer Response:

The interfacing could be improved, but we decided that the Swapper and the SCV are bound to each other, and the SCV effectively trusts the Swapper to check the tolerance.

# Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

## About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.